

PRACTICAL-05

AIM: Implementation of a knapsack problem using dynamic programming

PROGRAM:

```
#include<iostream>

using namespace std;

class Knapsack {
private:
    int w[10];      //weights
    int v[10];      // values
    int n;          // number of items
    int W;          // capacity
    int dp[10][100]; // dp table

public:
    // — Get Input —
    void getInput() {
        cout << "Enter number of items  : ";
        cin >> n;
        cout << "Enter knapsack capacity : ";
        cin >> W;
        for(int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : ";
            cin >> w[i] >> v[i];
        }
    }

    // — Build dp table —
    void solve() {
        // base case — row 0 and col 0
```

```
for(int i = 0; i <= n; i++) dp[i][0] = 0;
for(int j = 0; j <= W; j++) dp[0][j] = 0;

// fill table
for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= W; j++) {

        if(w[i] <= j) {
            // take or not take
            dp[i][j] = max(
                v[i] + dp[i-1][j-w[i]],
                dp[i-1][j]
            );
        } else {
            // cannot take item i
            dp[i][j] = dp[i-1][j];
        }
    }
}
}
```

```
// — Print dp table —
void printTable() {
    cout << "\nDP Table:" << endl;
    cout << "    ";
    for(int j = 0; j <= W; j++)
        cout << j << " ";
    cout << endl;

    for(int i = 0; i <= n; i++) {
```

```
    cout << "i=" << i << " ";
    for(int j = 0; j <= W; j++)
        cout << dp[i][j] << " ";
    cout << endl;
}
}

// — Find selected items —
void findItems() {
    cout << "\nSelected Items:" << endl;
    int j = W;
    for(int i = n; i >= 1; i--) {
        if(dp[i][j] != dp[i-1][j]) {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") TAKEN " << endl;
            j = j - w[i];
        } else {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") NOT taken " << endl;
        }
    }
}

// — Show result —
void showResult() {
    cout << "\nMaximum Value = "
        << dp[n][W] << endl;
```

```
}  
};  
  
// — MAIN —  
  
int main() {  
    Knapsack k;  
    k.getInput();  
    k.solve();  
    k.printTable();  
    k.showResult();  
    k.findItems();  
    return 0;  
}
```

OUTPUT:

```
Output  
  
Enter number of items : 4  
Enter knapsack capacity : 8  
Item 1 weight and value : 2  
1  
Item 2 weight and value : 3  
2  
Item 3 weight and value : 4  
5  
Item 4 weight and value : 5  
6  
  
DP Table:  
   0  1  2  3  4  5  6  7  8  
i=0  0  0  0  0  0  0  0  0  0  
i=1  0  0  1  1  1  1  1  1  1  
i=2  0  0  1  2  2  3  3  3  3  
i=3  0  0  1  2  5  5  6  7  7  
i=4  0  0  1  2  5  6  6  7  8  
  
Maximum Value = 8  
  
Selected Items:  
Item 4 (w=5, v=6) TAKEN  
Item 3 (w=4, v=5) NOT taken  
Item 2 (w=3, v=2) TAKEN  
Item 1 (w=2, v=1) NOT taken
```

CONCLUSION:

The 0/1 Knapsack problem was successfully implemented using Dynamic Programming. The algorithm finds the maximum value within the given knapsack capacity and identifies the selected items. Its best, average and worst-case time complexity is $O(nW)$.